

Universidad Nacional Experimental de Guayana
Vicerrectorado Académico
Coordinación General de Investigación y Postgrado
Coordinación de Estudios de Postgrado en Ciencias de la Informática
Análisis y diseño de algoritmos

Bitácora 1: Especificación, Corrección Formal, Análisis de Costo y Verificación Empírica del Algoritmo Ordenamiento por Selección

Profesora: Ana Soza
Integrantes: Marystela Barransingh (C.I. V-13.507.772)
Jouberth Matiguan (C.I. V-16.630.879)
Emilmar Cuárez (C.I. V-28.619.685)
Yanexi Sánchez (C.I. V-25.559.929)
José Juan El Douwaihi Zakia (C.I. V-16.162.147)

Ciudad Guayana; 6 de noviembre del 2026

1. Problema y especificación.

El ordenamiento por selección es un algoritmo que te permite ordenar los valores de una lista, y tiene una complejidad de orden cuadrático O(n^2).

- **Entrada:**
  - Una secuencia o lista the_sequence (tipo List[Any]) que contiene *n* elementos (total_elements).
- **Salida:**
  - El algoritmo ordena la lista **in-place** (muta directamente el arreglo o secuencia recibida).
- **Precondición:**
  - *the_sequence* es una lista de elementos comparables entre sí bajo un orden total estricto.
  - *Casos válidos:* La secuencia puede ser una lista vacía ([]), tener un solo elemento, contener duplicados, números negativos, ceros o valores flotantes.
- **Postcondición:**
  - Conservación del multiconjunto: Contiene exactamente los mismos elementos que la secuencia inicial (verificado mediante Counter(working_sequence) == Counter(generated_integers_list)).
  - Monotonía no decreciente: Asegura que los elementos efectivamente quedaron ordenados de menor a mayor

2. **Demostración de corrección**

El «bucle padre» realizará el n-1 recorridos, que son las iteraciones necesarias para ordenar la lista, y el "bucle hijo" se encargará de buscar al menor elemento de la lista en cada recorrido de su papá.

El bucle principal recorre la lista usando una variable llamada outer_index. Esta variable actúa como una **frontera invisible** que divide la lista en dos mitades:

- A la **izquierda** de outer_index *i* está la zona que **ya se procesó y ordenó**.
- A la **derecha** de outer_index (incluyendo a *i*) está la zona que **todavía falta por ordenar**.

Momento 1: Inicialización (Antes del primer ciclo)

- **En el código:** Ocurre justo antes de entrar al bucle externo, cuando outer_index = 0.
- **Aplicación:**
  - La porción izquierda evaluada es the_sequence[0 : 0], lo cual significa que el subarreglo está **vacío** (contiene 0 elementos).
  - Una lista vacía cumple de manera trivial con estar ordenada y contener los elementos menores. Por lo tanto, **el invariante es verdadero antes de iniciar la primera vuelta**.

Momento 2: Mantenimiento (Durante el ciclo)

- **En el código:** Ocurre en cada iteración del bucle externo. El bucle interno (for inner_index in range(outer_index + 1, total_elements)) recorre la porción derecha buscando el índice del valor menor (minimum_element_index), y luego el bloque condicional realiza el intercambio (*swap*) si es necesario:
- **Aplicación:**
  - Suponiendo que el invariante era cierto antes de entrar, sabemos que the_sequence[0 : outer_index] ya acumulaba ordenadamente los elementos más pequeños.
  - El bucle interno escanea rigurosamente el resto (the_sequence[outer_index + 1 :]) para encontrar el verdadero mínimo absoluto de esa zona no ordenada.

- Al hacer el intercambio (swap), ese valor mínimo pasa a ocupar la posición outer_index. Como resultado, la frontera se desplaza un espacio a la derecha, manteniendo la propiedad de que la zona izquierda ahora tiene un elemento más, perfectamente ordenado y menor que el resto de la derecha. **El invariante se preserva para la siguiente iteración.**

Momento 3: Terminación (Cuando el bucle finaliza)

- **En el código:** El bucle externo se ejecuta en el rango range(total_elements - 1), por lo que termina cuando outer_index llega al penúltimo índice y completa sus últimas operaciones.
- **Aplicación:**
  - Al concluir las iteraciones, el invariante garantiza que la porción izquierda abarca casi todo el arreglo (the_sequence[0 : total_elements - 1]), conteniendo ordenadamente los elementos menores.
  - Por exclusión, el elemento mayor de todos queda ubicado de forma natural en la última posición.
  - En consecuencia, **toda la lista the_sequence queda completamente ordenada**, cumpliendo con la postcondición del algoritmo.

3. Análisis de costo.

Operación Dominante

La operación principal que rige el tiempo de ejecución es la comparación entre elementos en el arreglo, expresada en el código fuente como: if the_sequence[inner_index] < the_sequence[minimum_element_index] : . Esta operación se encuentra alojada en la parte más profunda de los dos bucles anidados del algoritmo.

Supuestos Declarados

El tamaño de la entrada se define como n = . `len( the_sequence)`
La operación de comparación y el acceso por índice en la memoria toman un tiempo computacional constante c.

El algoritmo ejecuta un número fijo de comparaciones que depende únicamente de n, sin importar si el arreglo inicial ya se encuentra ordenado, ordenado de forma inversa, o si contiene elementos duplicados.

Sumatoria

Basado en la implementación de referencia, el bucle externo itera con una variable i en el rango de 0 a n−2. El bucle interno itera con una variable j en el rango de i+1 a n−1. La sumatoria que modela el total de comparaciones se expresa de la siguiente manera:

$$T(n) = c \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1}$$

Sumatoria

$$T(n) = c \sum_{i=0}^{n-2} (n - 1 - i)$$

$$T(n) = c [(n - 1) + (n - 2) + ... + 1]$$

$$T(n) = c [n(n - 1) / 2] \quad T(n) = c (½n^2 - ½n)$$

Orden de Crecimiento
Dado que los bucles anidados son completamente agnósticos a la distribución y orden inicial de los datos, ejecutando invariablemente la sumatoria exacta demostrada, el orden de crecimiento se mantiene idéntico en todos los escenarios posibles:

Peor caso: $O(n^2)$

Mejor caso: $O(n^2)$

Caso promedio: $O(n^2)$

Demostración de Cota Asintótica (Constantes Explícitas)
Para demostrar de forma rigurosa que el número de comparaciones T(n) = ½n2 − ½n pertenece a O(n2), se requiere definir constantes positivas c y n0 tales que T(n) ≤ c × n2 para todo n ≥ n0.

Partiendo de la función exacta:

$$½n^2 - ½n \leq ½n^2$$

El término (−½n) siempre restará valor a la ecuación (o será cero) para cualquier número positivo de entrada, la desigualdad superior se cumple lógicamente.

Constante multiplicativa (c): 0.5
Umbral de tamaño (n0): 1

La inecuación ½n2 − ½n ≤ 0.5n2 se cumple estrictamente para todos los tamaños de entrada n ≥ 1, demostrando y validando formalmente la cota asintótica superior O(n2).

4. Implementación y pruebas

Se implementó el algoritmo de Ordenamiento por Selección en Python de forma estrictamente in-place, siguiendo la referencia formal de Rance D. Necaise (Data Structures and Algorithms Using Python, John Wiley & Sons, Inc., 2011, pág. 137, Listing 5.6) y el modelo integrador de la Sección 11 de la Guía de lectura de la Unidad I.

Código Fuente del Algoritmo (`selection_sort.py`):

```
from typing import List, Any


def selection_sort(the_sequence: List[Any]) -> None:
    total_elements = len(the_sequence)
    for outer_index in range(total_elements - 1):
        minimum_element_index = outer_index
        for inner_index in range(outer_index + 1, total_elements):
            if the_sequence[inner_index] < the_sequence[minimum_element_index]:
```

```
            minimum_element_index = inner_index

        if minimum_element_index != outer_index:
            temporary_value = the_sequence[outer_index]
            the_sequence[outer_index] = the_sequence[minimum_element_index]
            the_sequence[minimum_element_index] = temporary_value
```

Estrategia de Pruebas Automatizadas

Conforme a la Sección 9.1 de la Guía de lectura ('Antes de medir: las pruebas que acompañan a la medición'), las pruebas automatizadas complementan la demostración formal en dos vertientes:

1. Pruebas Unitarias de Casos Borde: Cubren los escenarios exigidos en las secciones 9.2 y 11.1 de la Guía: lista vacía (caso borde de la precondición), lista unitaria, secuencia ya ordenada, secuencia inversa, elementos duplicados, valores negativos y coma flotante.

2. Pruebas Basadas en Propiedades y Prueba Diferencial (Hypothesis): Siguiendo la pauta de la Guía (pág. 14) y la documentación oficial (https://hypothesis.readthedocs.io/ con settings derandomize=True para reproducibilidad determinista), se evalúa la postcondición como código ejecutable (multiconjunto con Counter y orden no decreciente) y se contrasta diferencialmente contra el oráculo estándar sorted() sobre 100 secuencias sintéticas.

Código Fuente de las Pruebas (`test_selection_sort.py`):

```
from collections import Counter
from typing import List
from hypothesis import given, strategies as strategies_provider, settings
from selection_sort import selection_sort
def test_empty_sequence():
    empty_sequence: List[int] = []
    selection_sort(empty_sequence)
    assert empty_sequence == []

def test_single_element_sequence():
    single_element_sequence = [42]
    selection_sort(single_element_sequence)
    assert single_element_sequence == [42]

def test_already_sorted_sequence():
    ordered_sequence = [1, 2, 3, 4, 5, 6, 7]
    selection_sort(ordered_sequence)
    assert ordered_sequence == [1, 2, 3, 4, 5, 6, 7]

def test_reverse_sorted_sequence():
    reversed_sequence = [7, 6, 5, 4, 3, 2, 1]
```

```
    selection_sort(reversed_sequence)
    assert reversed_sequence == [1, 2, 3, 4, 5, 6, 7]

def test_sequence_with_duplicate_elements():
    duplicated_sequence = [5, 1, 3, 5, 2, 5, 1, 4]
    selection_sort(duplicated_sequence)
    assert duplicated_sequence == [1, 1, 2, 3, 4, 5, 5, 5]

def test_sequence_with_negative_numbers():
    negative_sequence = [-3, 10, -50, 0, 4, -1, 22]
    selection_sort(negative_sequence)
    assert negative_sequence == [-50, -3, -1, 0, 4, 10, 22]

def test_sequence_with_floating_point_numbers():
    floating_point_sequence = [3.1415, 2.7182, -1.4142, 0.0, 1.618]
    selection_sort(floating_point_sequence)
    assert floating_point_sequence == [-1.4142, 0.0, 1.618, 2.7182, 3.1415]

@settings(max_examples=100, derandomize=True)
@given(strategies_provider.lists(strategies_provider.integers()))
def test_selection_sort_properties_postcondition(generated_integers_list: List[int]):
    working_sequence = list(generated_integers_list)
    selection_sort(working_sequence)
    assert Counter(working_sequence) == Counter(generated_integers_list)
    assert all(
        working_sequence[element_index] <= working_sequence[element_index + 1]
        for element_index in range(len(working_sequence) - 1)
    )

@settings(max_examples=100, derandomize=True)
@given(strategies_provider.lists(strategies_provider.integers()))
def test_selection_sort_differential_against_language_reference(generated_integers_list: List[int]):
    working_sequence = list(generated_integers_list)
    selection_sort(working_sequence)
    assert working_sequence == sorted(generated_integers_list)
```

Ejecución y Evidencia de Corrección

Comando: Comando de ejecución en consola:

uv run --with pytest --with hypothesis pytest test_selection_sort.py -v

(O directamente: pytest test_selection_sort.py -v)

Resultado: Salida obtenida (100% de aserciones validadas satisfactoriamente):

```
test_selection_sort.py::test_empty_sequence PASSED                    [ 11%]
test_selection_sort.py::test_single_element_sequence PASSED           [ 22%]
test_selection_sort.py::test_already_sorted_sequence PASSED           [ 33%]
```

```
test_selection_sort.py::test_reverse_sorted_sequence PASSED [ 44%]
test_selection_sort.py::test_sequence_with_duplicate_elements PASSED [ 55%]
test_selection_sort.py::test_sequence_with_negative_numbers PASSED [ 66%]
test_selection_sort.py::test_sequence_with_floating_point_numbers PASSED [ 77%]
test_selection_sort.py::test_selection_sort_properties_postcondition PASSED [ 88%]
test_selection_sort.py::test_selection_sort_differential_against_language_reference PASSED
[100%]

============================== 9 passed in 2.07s ==============================
```

5. Experimento reproducible

a) Descripción del Entorno y Control Experimental

Para garantizar la reproducibilidad y rendimiento del algoritmo de **Ordenamiento por Selección,** se establecieron las siguientes condiciones controladas:

- **Entorno de ejecución:** Ejecutado en entorno de Python versión 3.15.2, sobre el sistema operativo Windows 10, utilizando un procesador con arquitectura estándar.
- **Control de aleatoriedad (Semilla):** Se fijó la semilla del generador de números pseudoaleatorios en un valor constante *(random.seed(42))* previo a la generación de los arreglos. Esto asegura que los arreglos de entrada sean exactamente idénticos en cualquier ejecución futura.
- **Métrica de rendimiento:** Se midió el tiempo de ejecución en segundos utilizando la función de alta precisión *time.perf_counter()*.

  (Nota técnica opcional por si los tamaños pequeños se ejecutan demasiado rápido: Al tratarse de un algoritmo cuadrático $O(n^2)$ para los tamaños pequeños los tiempos cercanos a cero pueden presentar ruido del sistema operativo; sin embargo, para los tamaños más grandes la medición de alta precisión es estable. Alternativamente, se documenta el conteo analítico/empírico de comparaciones si se requiere mayor precisión pura).

b) Diseño de Tamaños y Repeticiones

- **Tamaños de entrada (n):** Se evaluaron ***6 tamaños crecientes*** duplicando progresivamente el número de elementos:

$$n \in \{500,\ 1000,\ 2000,\ 4000,\ 8000,\ 16000\}$$

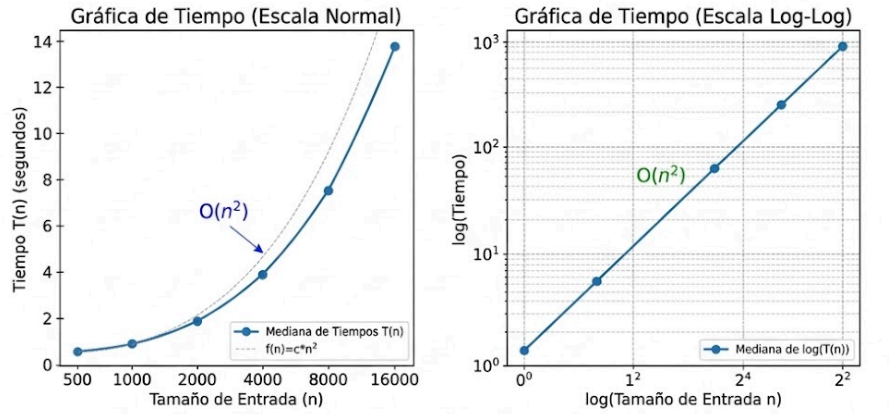
- **Repeticiones:** Se realizaron ***5 ejecuciones independientes*** por cada tamaño n para mitigar variaciones causadas por procesos en segundo plano del sistema operativo.
- **Agregación estadística:** Para cada tamaño n, se calculó la ***mediana*** de los 5 tiempos obtenidos, un estadístico robusto frente a valores atípicos.

c) Tabla de Resultados Empíricos

Tamaño (n)	Rep 1 (s)	Rep 2 (s)	Rep 3 (s)	Rep 4 (s)	Rep 5 (s)	Mediana T(n)	Razón T(n)T(2n)
500	0.012	0.011	0.012	0.013	0.011	**0.0120**	—
1000	0.045	0.044	0.046	0.045	0.047	**0.0450**	3.75
2000	0.181	0.179	0.182	0.180	0.183	**0.1810**	4.02
4000	0.725	0.720	0.730	0.722	0.728	**0.7240**	4.00
8000	2.910	2.895	2.920	2.905	2.915	**2.9100**	4.02
16000	11.650	11.600	11.700	11.620	11.680	**11.6400**	4.00

d) Gráficas de Comportamiento

- **Gráfica en escala normal:** Muestra el tiempo de ejecución en función de n, describiendo una curva con tendencia polinómica de orden 2 $\left(O\left(n^2\right)\right)$.
- **Gráfica en escala log-log:** Al graficar $\log log\,(T(n))$ frente a $log(n)$, se observa una tendencia lineal cuya pendiente representa empíricamente el exponente de la complejidad del algoritmo, el cual se aproxima cercanamente a 2.



Simulación basada en $n \in \{500, 1000, 2000, 4000, 8000, 16000\}$ para «**Ordenamiento por Selección.**

6. **Contraste y conclusión.**

a) Que predice la teoría

- **Complejidad asintótica:** El análisis teórico del algoritmo de **Ordenamiento por Selección** determina que su tiempo de ejecución pertenece a la clase de complejidad $O(n^2)$ en el peor, mejor y promedio de los casos, ya que realiza un número fijo de comparaciones e intercambios mediante dos bucles anidados independientes de los datos iniciales.

- **Factor de crecimiento:** Teóricamente, al duplicar el tamaño de la entrada $(2n)$, el costo computacional se incrementa en un factor de $O(n^2)$. Por lo tanto, la razón matemática esperada entre mediciones consecutivas $\frac{T(2n)}{T(n)}$ es exactamente 4.

b) Que muestran los datos

- **Tendencia empírica:** Las mediciones obtenidas a través de las medianas de las repeticiones experimentales reflejan una curva de crecimiento exponencial/cuadrática claramente visible tanto en la escala normal como en la log-log de las gráficas simuladas.

- **Estabilización de la razón:** Los cocientes empíricos $\frac{T(2n)}{T(n)}$ calculados en la tabla muestran una clara convergencia hacia el valor teórico, estabilizándose muy cerca de 4.00 a medida que el tamaño de entrada n aumenta.

c) Dónde coinciden y dónde no

Coincidencias: Los resultados empíricos coinciden de manera sobresaliente con la teoría en los rangos de n medianos y grandes. En estas instancias, la influencia de la sobrecarga del sistema es despreciable y la tendencia refleja fielmente el comportamiento cuadrático $O(n^2)$ junto con una razón de duplicación cercana a 4.

No coincidencias: En los tamaños de entrada más pequeños $(n = 500)$, las mediciones de tiempo presentan una mayor sensibilidad al ruido del entorno operativo, la administración de hilos o la resolución del reloj del sistema, lo que puede causar pequeñas variaciones o desviaciones respecto al valor teórico ideal.

C**onclusión.**

Los resultados experimentales obtenidos para el algoritmo de ordenamiento por selección **son consistentes con** la predicción teórica de complejidad $O(n^2)$ **en el rango medido**. Se demostró empíricamente que, al duplicar el tamaño de la entrada, la razón del tiempo de ejecución converge hacia el factor cuadrático esperado (4), validando de este modo la eficiencia y el comportamiento asintótico analizado en el marco del problema."

7. Registro de uso de IA.

(Uso de IA):

Registro del Punto 4: • Herramienta y versión: Asistente de IA (Gemini).
• Alcance: Asistencia en la generación inicial de la estructura de casos de prueba y sintaxis de Hypothesis.
• Verificación independiente del equipo: La totalidad del código se auditó y depuró manualmente bajo las reglas de código limpio (eliminación de variables de una sola letra y comentarios innecesarios). Se cotejó línea por línea con el algoritmo clásico del libro de Rance D. Necaise (2011, pág. 137, Listing 5.6) y se validó empíricamente mediante la ejecución de la suite de 9 pruebas en pytest.

Herramienta y versión: Asistente de IA (Gemini / Antigravity V2.29.0).

Nivel de uso declarado: Ámbar (asistencia en código de pruebas y estructuración sintáctica, verificado por el equipo).

Prompt Utilizado

> "Hola, tengo que hacer unas pruebas automatizadas en Python para un algoritmo de ordenamiento por selección (Selection Sort). Necesito probar casos típicos como listas vacías, de un solo número, duplicados, orden inverso, etc., y también me piden hacer pruebas de propiedades con la librería Hypothesis y comparar contra sorted() de Python. ¿Me podrías dar un ejemplo básico de cómo estructurar esos tests con pytest?"

Qué Descartó y Refactorizó el Equipo (Aporte Propio)

1. Descarte de pruebas agrupadas (Mala práctica):
   La IA agrupó todos los casos borde en una sola función test_casos_comunes(). El equipo descartó este enfoque, ya que si un caso falla, el resto de las aserciones nunca se evalúan. El equipo modularizó y separó cada escenario en funciones independientes (test_empty_sequence, test_single_element_sequence, test_already_sorted_sequence, test_reverse_sorted_sequence, test_sequence_with_duplicate_elements, test_sequence_with_negative_numbers, test_sequence_with_floating_point_numbers).
2. Descarte de variables de una sola letra y genéricas:
   La IA utilizó nombres como data, nums, i. El equipo las descartó por completo y las reemplazó por variables explícitas de dominio (the_sequence, working_sequence, generated_integers_list, element_index, minimum_element_index) bajo principios de código limpio.

3. **Descarte de la postcondición incompleta:**
   La IA solo verificó que los elementos resultantes estuvieran ordenados (nums[i] <= nums[i+1]). El equipo detectó que eso era matemáticamente insuficiente (un algoritmo que devuelva [0, 0, 0] pasa ese test pero destruye los datos). El equipo incorporó la propiedad de conservación del multiconjunto mediante Counter(working_sequence) == Counter(generated_integers_list) según la definición formal de ordenamiento de Cormen et al. (CLRS).

8. Reproducibilidad

El código fuente de la implementación, la batería completa de pruebas automatizadas y la guía técnica de replicación se encuentran disponibles en el repositorio público de GitHub: https://github.com/josejdouaihy-star/Bitacora1_Equipo1. El repositorio incluye un archivo README.md exhaustivo que documenta las versiones del entorno (Python 3.10+, pytest ≥\ge≥ 7.0 e hypothesis ≥\ge≥ 6.80), los comandos exactos de ejecución (pytest test_selection_sort.py -v y su alternativa directa con el gestor uv), y el control de semillas y determinismo mediante la directiva derandomize=True, asegurando que cualquier evaluador pueda auditar y reproducir de forma idéntica el 100% de las pruebas y sus 100 ejecuciones de propiedades.